

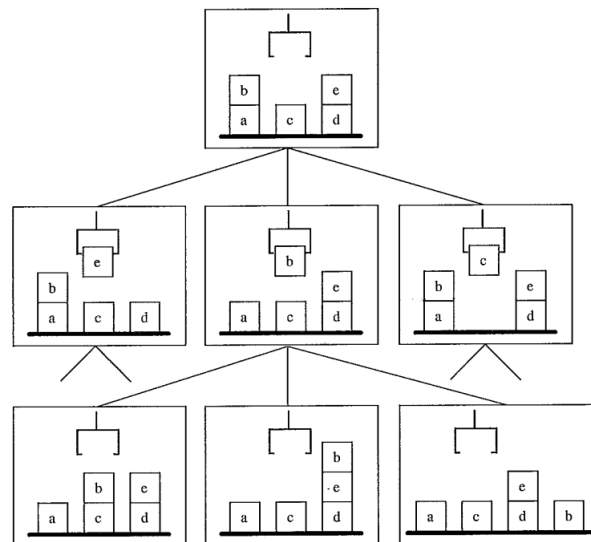
PART 1: Conceptual

Explain informally but clearly and precisely.

- Closed world assumption.
- The frame problem. Provide two examples.
- The three components an action using the STRIPS language? Explain, in your own words, each each of the components are meant to define and capture. Give an example of a STRIPS action.
- At least 2 limitations of STRIPS language to represent actions.
- The differences and similarities between problem solving/search and planning.
- What is the difference between path/motion planning and automated planning.
- Explain what GraphPlan is and what is it used for.

PART 2: Blocks World

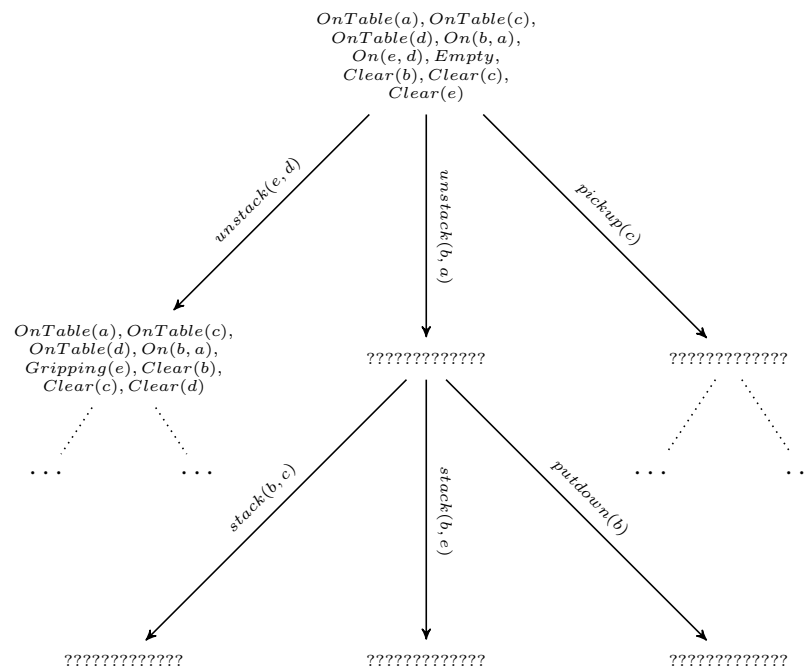
Consider the following blocks world scenario:



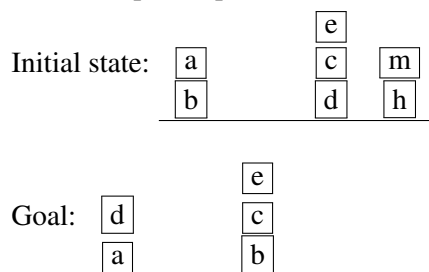
Suppose we use the following fluents (predicates in logic) to represent a states:

- $OnTable(x)$: block x is on the table.
- $On(x, y)$: block x is on top of block y .
- $Clear(x)$: there is nothing on top of block x .

- *Gripping(x)*: the gripper is holding block *x*.
 - *Empty*: the gripper is not holding anything.
- Specify the initial state in logical representation, that is, using conjunction of atoms and relying on the close world assumption.
 - Write the STRIPS representation for operators *pickup(x)*, *putdown(x)*, *stack(x, y)*, and *unstack(x, y)*. Should a block be made clear stacking or putting it down?
 - What is the new state if we apply operator-action *unstack(e, d)* to the initial state.
 - Explain in English what frame axioms would be required in this domain. Explain how those frame axioms were respected in the question before when *unstack(e, d)* was executed.
 - Use the operators (and the implicit frame axioms) of the previous question to complete (by filling ?????) part of the search state space of the blocks world given in the figure above. The root of the tree should be the logical initial state representation and edges should represent application of ground operators.



- Show an optimal plan that solves this problem:



- How many solutions are there for the above question? How many optimal solutions?
- Is the goal state correctly defined? Explain either way.
- What would GraphPlan give as heuristic value for the initial state? To find out, build the planning graph starting from the initial state. Is the value of the heuristic less, equal, or greater than the true length of the plan you found in the previous question? Explain why.
- Suppose that each block has a new property of colour. All the rest of the domain remains the same. What would you change to your representation and corresponding STRIPS modeling?

- (k) Suppose you want to represent the action of dropping an object. Its effect is that the object is on the floor and depending on whether it is fragile or not, it may or may not be broken. Can you represent such action in STRIPS representation? Either provide the representation or explain why not. Can you think of a way that you could work around this limitation by defining multiple actions?

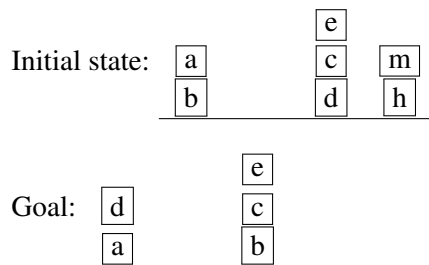
PART 3: Time to code in PDDL!.....

In this question, you will code a problem in PDDL and let the automated planning system find the solution for you! To do so, you will use the [planning.domain](#)'s editor, where you can code your domain and a specific problem and use an online solver to get the plan (if any exists!).

Check the video that I did [here](#) to show you how to use [planning.domain](#)'s editor and its animation facility!

For PDDL, besides the book, you may want to check the PDDL quick tutorial notes [here](#) and [here](#). Another useful material is this [PDDL by Example](#) pack of slides. Finally a nice web-based PDDL reference can be found [here](#).

- (a) First model the Blocks World domain we have seen above. Take the template file [domain.pddl](#) encoding the domain in PDDL and fill the gaps in the editor. You can just drag and drop the file and it will load it. Remember that the domain file only contains the fluents and operators. It is meant to be usable with any specific problem instance.
- (b) Next, encode the *specific problem* we saw above by completing the following template [problem.pddl](#) file. A problem file specifies what domain the problem is based on, the actual objects in the problem (in our case, what blocks there are), the initial state, and the condition for a state to be considered a goal state.



- (c) Find a solution by hitting the button **Solve!** in the web interface. If your domain and problem files are OK, you should get the plan with 10 actions!
- (d) Finally, animate it using the *Planimation* plugin using the [blockworld.AP.pddl](#) animation file.
- (e) Are you still curious? Do you want to see more involved domains with larger plans? Check the **parcprinter** domain, from a real-world case:

This domain models the operation of the multi-engine printer, for which one prototype is developed at the Palo Alto Research Center (PARC). This type of printer can handle multiple print jobs simultaneously. Multiple sheets, belonging to the same job or different jobs, can be printed simultaneously using multiple Image Marking Engines (IME). Each IME can either be color, which can print both color and black&white images, or mono, which can only print black&white image. Each sheet needs to go through multiple printer components such as feeder, transporter, IME, inverter, finisher and need to arrive at the finisher in order. Thus, sheet $(n + 1)$ needs to be stacked in the same finisher with sheet n of the same job, but needs to arrive at the finisher right after sheet n (no other sheet stacked in between those two consecutive sheets). Since IMEs are heterogeneous (mixture of color and mono) and can run at different speeds, optimizing the operation of this printer for a mixture of print jobs, each of them is an arbitrary mixture of color/b&w pages that are either simplex (one-sided print) or duplex (two-sided print) is a hard problem.

Use [this already prepared session](#) to solve one problem instance. How many nodes did the planner expanded? How many did it generate but did not manage to expand? Post those stats in the forum!

End of tutorial worksheet